AI Assisted Lead Qualification

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF ENGINEERING

By

Kunal Redij : 23106067

Soham Shelatkar: 23106023

Varun Panchal : 23106050

Makarand Panchal: 23106065

Under the Guidance of

Prof. Rashmi Dhake

Department of CSE (AI&ML)

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "AI Assisted Lead Qualification System" is a bonafide work of **Kunal Redij (23106115), Soham Shelatkar(), Varun Panchal (23106050), Makarand Panchal()** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (AI&ML)**

(Name and Sign)	(Name and Sign)
Guide	Project Co-ordinator
(Name and Sign)	(Name and Sign)
Head of Department	Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for T.E.

This project report entitled **AI Assisted Lead Qualification** System by ***Kunal Redij, Soham Shelatkar, Varun Panchal, Makarand Panchal*** is approved for the degree of ***Bachelor of Engineering*** in **Computer Science & Engineering (AI&ML)**, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Kunal Redij)

(Soham Shelatakar)

(Varun Panchal)

(Makarand Panchal)

Date:

Abstract

In modern businesses, a large volume of inbound leads is generated daily through websites, email campaigns, social media, and third-party platforms. Manually screening, scoring, and following up on these leads is time-consuming, inconsistent, and often results in missed opportunities. Sales teams spend a disproportionate amount of their productive time on low-priority or unqualified leads, leading to reduced conversion rates and poor resource utilization.

This project, titled "AI-Assisted Lead Qualification & Automated Follow-Up System," proposes an intelligent backend solution that automates the lead qualification process using Natural Language Understanding (NLU) and a hybrid AI + rule-based decision engine. The system analyzes incoming lead data, assigns a qualification score (High, Medium, or Low), detects intent and urgency signals, and manages lead lifecycle states (New → Qualified → Follow-up → Replied). Based on the qualification, the system automatically generates personalized, context-aware follow-up messages using a Large Language Model (LLM), ensuring timely and relevant outreach without human intervention.

The proposed architecture combines structured rule-based scoring with AI-powered intent detection to maintain explainability and reduce the risk of over-automation. By implementing lifecycle state management, the system prevents duplicate or spammy follow-ups and provides a clear audit trail for each lead. The solution is designed to be scalable, modular, and integrable with existing CRM tools and email platforms. This work aligns with UN Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by democratizing intelligent automation for small and medium enterprises, enabling resource-efficient sales operations and reducing barriers to advanced business technology. Experimental results demonstrate improved lead prioritization, reduced manual effort, and higher engagement rates compared to traditional static follow-up systems.

CONTENTS

Introduction 1

Literature Survey/ Existing system 3

2.1 Key Observations from Literature 4

Limitation of Existing system 7

3.1 Overview of Existing Approaches 7

3.2 Identified Limitations 8

Problem Statement and Objective 9

4.1 Problem Statement 9

4.2 Objectives 10

4.3 Scope 10

Proposed System 12

5.1 Analysis of the Problem Domain 12

5.2 System Framework 14

5.3 Algorithm 18

5.4 Design Details 24

5.5 Methodology 28

Experimental Setup 33

6.1 Details of Database and Input 33

6.2 Performance Evaluation Parameters (Validation) 37

6.3 Development Environment 44

Results and Discussion 45

7.1 Strengths 45

7.2 Threats to Validity 45

7.3 Future Improvements 47

7.4 Demonstration 48

Conclusion and Future Work 49

8.1 Conclusion 49

8.2 Future Scope 50

References 51

LIST OF FIGURES

Figure 1:System Architecture..... 15

Figure 2:Lead Qualification Pipeline 18

Figure 3:State Machine Diagram 22

Figure 4:Validation Rules Diagram 35

Figure 5:Intent Classification Confusion Matrix Heatmap 37

Figure 6: Lead Classification Confusion Matrix..... 39

Figure 7:API Latency Percentile Graph 41

Figure 8: Performance Metrics Bar Chart 42

Figure 9:Lead Dashboard..... 48

LIST OF TABLES

Table 1:Literature Survey..... 4

Table 2:Key observation Table 5

Table 3:Lead Table Schema..... 33

Table 4:Intent Classification Table 38

Table 5:System Performance Metric Table..... 40

Table 6:Business Impact Metrics Table 41

Table 7:Error Analysis Table 43

Table 8:Development Environment Table 44

ABBREVATION

AI: *Artificial Intelligence*

ML: *Machine Learning*

CNN: *Convolutional Neural Networks*

RAG: *Retrieval-Augmented Generation*

BANT: *Budget, Authority, Need, and Timeline*

NLP: *Natural Language Processing*

NLU: *Natural Language Understanding*

LLM: *Large Language Model*

CRM: *Customer Relationship Management*

BERT: *Bidirectional Encoder Representations from Transformers*

LSTM: *Long Short-Term Memory*

SME: *Small and Medium Enterprises*

REST: *Representational State Transfer*

Chapter 1

Introduction

In the contemporary digital economy, organizations generate a high volume of potential customer inquiries through websites, email campaigns, social media, paid advertisements, and partner platforms. Although this creates abundant opportunity, it also introduces a practical challenge: not all leads carry equal purchase intent. Sales teams must rapidly determine which prospects are ready to engage, which require nurturing, and which should be deprioritized. This process, known as lead qualification, is central to pipeline quality, revenue conversion, and overall sales efficiency.

In many organizations, lead qualification remains largely manual. Sales development representatives examine incoming messages, evaluate prospect details against business criteria such as Budget, Authority, Need, and Timeline (BANT), and then route promising leads for deeper engagement. While effective at small scale, this approach becomes difficult to sustain as lead volume grows. Manual review is time-consuming, reviewer-dependent, and susceptible to inconsistency and bias. More importantly, delayed response to inbound interest reduces conversion likelihood; industry evidence consistently shows that shorter first-response times produce significantly better outcomes.

A second bottleneck appears after qualification: follow-up communication. Even when a lead is correctly prioritized, teams often struggle to send responses that are both timely and personalized. In practice, many workflows rely on static templates with minimal contextual adaptation, resulting in lower engagement and missed opportunities. Therefore, the challenge is not only to classify leads accurately, but also to generate relevant follow-up communication that reflects prospect intent, urgency, and context.

Recent advances in Artificial Intelligence, especially Natural Language Processing, large language models, and machine-learning classification, provide a strong foundation to address these limitations. AI systems can analyze unstructured lead text, detect intent and urgency signals, assign confidence-based scores, and generate context-aware follow-up drafts at

operational scale. When integrated within a controlled backend workflow, these capabilities can reduce manual effort, improve scoring consistency, and accelerate lead response cycles.

This project, titled AI-Assisted Lead Qualification and Automated Follow-Up System, proposes a unified backend framework that ingests inbound lead data, applies a hybrid AI plus rule-based qualification engine, manages lead lifecycle states, and supports automated follow-up generation. The design emphasizes explainability and governance so that model outputs are traceable, reviewable, and adjustable according to business policy. Beyond technical contribution, the work aligns with United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by enabling practical intelligent automation for scalable, resource-efficient sales operations, particularly for small and medium enterprises.

Alignment with SDG 9: Industry, Innovation and Infrastructure:

This project directly supports United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by democratizing access to intelligent automation capabilities that have traditionally been available only to large enterprises with substantial technical resources. By providing an open, scalable AI-assisted lead qualification framework, the system enables small and medium enterprises (SMEs) to compete more effectively in digital markets, reducing operational inefficiencies and resource waste inherent in manual lead processing.

Furthermore, the emphasis on explainability, governance, and hybrid rule-based approaches ensures that AI adoption remains transparent and controllable, fostering resilient infrastructure for responsible automation. In resource-constrained contexts, where every salesperson's time is precious, this system maximizes conversion efficiency without proportional increases in labor or cost, thereby contributing to sustainable economic growth and technological capability-building across diverse organizational scales.

Chapter 2

Literature Survey/ Existing system

The field of AI-driven sales automation and lead qualification has seen growing interest over the past decade, with contributions from both academic research and industry. The following review covers key works relevant to automated lead scoring, intent detection using NLP, CRM automation, and LLM-based communication generation.

Author / Year	Title / Source	Key Findings	Limitations Addressed
Salesforce Research, 2023	State of Sales Report 2023	67% of sales reps time is spent on non-selling tasks; AI tools improve lead conversion by 30%.	Highlights need for automated lead scoring to free up human effort.
Xu et al., 2023	Deep Learning-based Lead Scoring	LSTM model achieves 88% accuracy in predicting lead	Static rule-based scoring systems lack adaptability.

	in CRM Systems – IEEE Access	conversion; behavioral signals outperform demographic data.	
Chung & Kim, 2022	NLP-Driven Intent Classification for B2B Sales – ACM SIGKDD	BERT-based model classifies sales intent with 91% F1-score from email/chat logs.	Manual labeling of intents is costly and slow.
Paolanti et al., 2022	AI in CRM: Automated Pipeline Management – Springer ICCS	Hybrid AI pipeline reduces lead response time by 45% and improves funnel throughput.	No lifecycle state management in prior systems.
Vaessen et al., 2021	GPT-based Automated Email Personalization for Sales Outreach – arXiv	GPT-3.5 generated follow-ups show 23% higher open rate vs. templated emails.	Prior tools use static templates; lack context-aware generation.

Table 1:Literature Survey

2.1 Key Observations from Literature

The surveyed literature consistently highlights two core challenges: (1) the inability of static scoring models to adapt to dynamic buyer behavior, and (2) the lack of structured lifecycle management in existing automated systems. While individual contributions address NLP-based intent classification, probabilistic lead scoring, and LLM-based email generation separately, no unified end-to-end system combining all three components with explicit state management has been proposed for small-to-medium enterprise use cases. This gap forms the primary motivation for the proposed system.

Feature	SF Einstein	HS Score	Xu (2023)	Vaessen	This Work
Intent Detection	✓	✗	✓	✗	✓
Scoring	ML	Rules	LSTM	✗	Hybrid
Follow-up	✗	Templates	✗	✓ (GPT)	✓ (Llama)
State Management	✗	✗	✗	✗	✓

Explainable	✗	Partial	✗	✗	✓
Human-in-Loop	✗	✗	✗	✗	✓
Accessibility	✗	✗	Research	Research	✓

Table 2:Key observation Table

2.1.1 Evolution from Rule-Based to AI-Driven Scoring

Early lead qualification systems, deployed in CRM tools like Salesforce and HubSpot from the mid-2000s onward, relied on purely rule-based scoring: assigning fixed point values to demographic attributes such as industry, company size, and job title. While deterministic and easy to audit, these systems were fundamentally static. They could not adapt to shifts in buyer behavior, respond to contextual signals in lead messages, or improve over time without manual reconfiguration. Researchers such as Xu et al. (2021) demonstrated that behavioral and textual signals from lead communications — including message length, sentiment polarity, and keyword frequency — are far more predictive of conversion than demographic features alone. Their LSTM-based model, trained on CRM interaction logs, achieved an 88% classification accuracy, substantially outperforming demographic-only baselines. This finding has since been replicated across multiple studies and represents a consensus shift in the field: intent, expressed through language, is the strongest predictor of lead quality.

2.1.2 Role of Transformer Models in Intent Detection

The advent of transformer-based language models, beginning with the publication of BERT by Devlin et al. (2019), fundamentally changed what is possible in NLP-driven sales automation. Prior to BERT, intent classification systems used bag-of-words representations or shallow recurrent networks that struggled with long-range dependencies, negation, and domain-specific vocabulary. BERT's bidirectional attention mechanism enabled models to understand context at a sentence and document level, dramatically improving classification accuracy for ambiguous orpolysemous expressions common in sales communication (e.g., "we are looking at options" could signal comparison shopping or active purchase intent depending on context). Chung & Kim (2022) fine-tuned a BERT model on a proprietary dataset of 12,000 annotated B2B sales emails and achieved an F1-score of 0.91 across five

intent categories. Their work also highlighted an important practical finding: domain-specific fine-tuning on even a modest dataset (3,000–5,000 labeled examples) yields substantially better results than using a general-purpose model out of the box. This directly informs the design decision in the proposed system to fine-tune DistilBERT on a sales-specific intent corpus rather than using zero-shot classification. DistilBERT was chosen over full BERT for its 40% smaller footprint and 60% faster inference speed, with only a marginal accuracy reduction — a practical trade-off that is particularly important for a backend system expected to process hundreds of leads per minute.

2.1.3 Hybrid AI + Rule-Based Architectures

A recurring theme across the literature is the tension between AI-driven flexibility and rule-based explainability. Pure AI systems, while accurate, are often described as "black boxes" that cannot provide human-interpretable justifications for their decisions. In a sales context, this is a critical limitation: sales managers need to understand and trust the scoring system, and sales representatives need to be able to override or escalate individual decisions. Studies by Paolanti et al. (2022) demonstrated that a hybrid approach — where rule-based criteria handle clear-cut cases (e.g., a decision-maker from a Fortune 500 company explicitly requesting a demo) while AI handles ambiguous cases — achieves better overall performance than either approach alone. Their hybrid pipeline reduced lead response time by 45% and improved funnel throughput compared to the manual baseline, while maintaining a degree of explainability through the rule component.

This insight is central to the architecture of the proposed system. The qualification engine uses a weighted combination of a configurable YAML-based rule module (covering BANT criteria) and an NLU-derived intent score. The rule component ensures that obvious high-priority leads are never missed due to model uncertainty, while the AI component captures subtle signals

2.1.4 LLM-Based Communication Generation in Sales

The application of large language models to sales communication represents one of the most recent and rapidly evolving areas of the field. Vaessen et al. (2023) conducted a controlled study comparing GPT-3.5-generated follow-up emails to standard templated emails used by a B2B SaaS company over a 90-day period. The LLM-generated emails, constructed using a dynamic prompt that incorporated the lead's stated problem, industry, and engagement history, achieved a 23% higher open rate and a 17% higher reply rate compared to the template baseline. These results were statistically significant ($p < 0.01$) and held across multiple industry segments.

However, the same study also identified risks associated with unconstrained LLM generation: approximately 8% of generated emails contained factual claims about the product that were inaccurate, and 3% had a tone deemed inappropriate by human reviewers. These findings strongly motivate the inclusion of a content guardrail module in the proposed system — a rule-

based post-processing step that validates the generated email for length, tone, factual consistency with the product knowledge base, and compliance with anti-spam regulations before it is stored for dispatch.

2.1.5 Research Gap: End-to-End Systems with State Management

A systematic review of the literature reveals a clear and persistent gap: while individual components of the lead qualification pipeline have been studied extensively, no published work proposes or evaluates a unified end-to-end system that integrates all four capabilities — NLU-based intent detection, hybrid qualification scoring, lifecycle state management, and LLM-based follow-up generation — within a single, deployable backend architecture. Existing commercial tools (Salesforce Einstein, HubSpot AI Score) address scoring but not follow-up generation. Research tools address individual NLP tasks but are not packaged as deployable.

Chapter 3

Limitation of Existing system

3.1 Overview of Existing Approaches

Several commercial CRM platforms, including Salesforce, HubSpot, and Zoho, provide lead scoring and workflow automation capabilities. In recent years, these platforms have also incorporated AI-assisted features such as predictive scoring, email drafting, and engagement recommendations. While these developments represent meaningful progress, most current solutions still fall short of delivering a fully automated, explainable, and state-aware lead qualification pipeline. In practice, many implementations remain fragmented across scoring, follow-up generation, and lifecycle tracking, which limits reliability and operational transparency.

3.2 Identified Limitations

1. Heavy dependence on pure LLM decisions

   Many AI-enabled sales tools rely predominantly on large language models for scoring and messaging, without sufficient rule-based constraints. This can produce inconsistent outputs, hallucinated assessments, and follow-up content that may violate business policy or compliance expectations. Such approaches also reduce auditability.
2. Lack of structured lifecycle state management

   Existing systems often store leads as static records rather than managing explicit state transitions (e.g., New, Qualified, Follow-up Sent, Replied, Closed). This increases the risk of duplicate outreach, weakens follow-up traceability, and reduces visibility into pipeline progression.
3. Inconsistent intent scoring

   A significant number of tools prioritize demographic attributes (company size, industry, designation) over behavioral and textual intent signals present in communications. As a result, high-intent leads may be undervalued, while low-intent leads from large organizations may be over-prioritized.
4. Static follow-up messaging

   Most automated follow-up workflows still rely on template-based emails with minimal personalization. Simple variable substitution (name, company) rarely addresses lead-specific questions, objections, or urgency, resulting in lower engagement and weaker response quality.
5. Risk of spam-like automation

   Without state-aware logic, response detection, and pacing controls, systems may send repeated messages to the same lead in short intervals. This can harm brand perception and create regulatory risk under frameworks such as CAN-SPAM and GDPR.
6. Limited explainability

   Sales leaders and enterprise buyers require clear justification for lead prioritization decisions. Black-box scoring systems that provide no reasoning or feature-level signal visibility are difficult to validate, debug, and improve over time.
7. Poor fit for small and medium businesses

   Many advanced AI qualification suites are designed for large enterprises with high integration budgets and dedicated IT teams. SMEs often need lightweight, API-first

solutions that can integrate quickly with email workflows, spreadsheets, and compact CRM stacks.

8. Weak handling of unstructured data

   Current systems frequently underperform on long-form emails, conversational text, mixed-format inputs, and multilingual content. This leads to intent misinterpretation and unreliable lead scoring in real-world communication settings.

Chapter 4

Problem Statement and Objective

4.1 Problem Statement

Businesses struggle to efficiently manage and prioritize inbound leads due to manual processes, delayed responses, and lack of structured tracking. Manual qualification is inconsistent, time-consuming, and fails to scale as lead volumes grow. Traditional automation tools either require heavy manual configuration or rely entirely on AI without proper safeguards, auditability, or lifecycle management.

Specifically, the following problems are addressed:

- Delayed lead response due to manual review queues, reducing conversion probability.
- Inconsistent qualification criteria across different sales representatives.
- Static follow-up messages that fail to engage leads meaningfully.
- Absence of lead lifecycle tracking, leading to duplicate outreach and poor pipeline visibility.

- Lack of explainability in AI scoring, preventing trust and adoption by sales teams.

4.2 Objectives

1. Design and implement an automated lead qualification engine that scores incoming leads as High, Medium, or Low priority using a hybrid AI + rule-based approach.
2. Develop an NLU-based intent and urgency detection module that extracts actionable signals from lead messages.
3. Implement a lead lifecycle state management system that tracks lead progression from New → Qualified → Follow-up → Replied.
4. Build an LLM-powered follow-up generation module that creates personalized, context-aware follow-up messages based on lead data and qualification output.

5. Ensure explainability by providing human-readable reasoning for each qualification decision.
6. Design a modular, API-driven backend that can integrate with existing CRM and email platforms.
7. Validate the system through testing on sample lead datasets, measuring qualification accuracy and follow-up engagement quality.

4.3 Scope

The scope of this project is limited to the backend system for lead qualification and follow-up generation. The following are explicitly within scope:

- Processing of structured lead data (name, company, email, message) from API inputs.
- NLU-based intent and urgency classification using pretrained models.
- Rule-based and AI-assisted lead scoring.
- Automated follow-up email generation using an LLM.
- Lead state management using a lightweight database.

The following are out of scope for this mini project:

- A full frontend CRM dashboard (UI mockups may be provided for demonstration).
- Direct email server integration and actual email dispatch.
- Multi-language support for non-English leads.
- Real-time integration with live CRM platforms such as Salesforce or HubSpot.

Chapter 5

Proposed System

5.1 Analysis of the Problem Domain

The problem domain of automated lead qualification emerges from a common limitation in modern sales operations: inbound leads are generated faster than they can be evaluated manually. In many organizations, lead handling still depends on sales representatives reading inquiries one by one, judging intent from limited context, and deciding whether to respond immediately, defer the lead, or ignore it. Although this process may work for low volumes, it becomes inefficient and inconsistent as lead volume grows. The result is a clear gap between the speed of incoming opportunities and the capacity of human teams to process them effectively.

5.1.1 Manual Qualification Challenges

Manual lead qualification presents several operational difficulties. The first and most visible challenge is response delay. A lead that is not reviewed quickly may lose interest or move to a competitor before any meaningful follow-up occurs. In practice, the time between receiving a lead and assigning it to a sales owner can vary significantly depending on workload, availability, and individual judgment. This delay directly reduces the probability of conversion because sales opportunities are often time-sensitive.

A second challenge is inconsistency in judgment. Different sales representatives may evaluate the same lead differently depending on experience, familiarity with the product, or subjective interpretation of the message. One representative may classify a lead as highly qualified based on urgency cues, while another may treat the same lead as a low-priority inquiry. This lack of consistency weakens the reliability of the qualification process and makes performance tracking difficult.

A third challenge is workload overload. Sales teams often receive more inquiries than they can handle manually, especially when marketing campaigns, website forms, and outbound channels are active at the same time. Instead of focusing on meaningful conversations with high-potential prospects, representatives spend considerable time sorting messages, extracting intent, and drafting repetitive responses. This creates inefficiency and reduces productivity across the team.

Manual qualification also suffers from limited scalability. As lead volume increases, the number of people required to maintain fast and accurate processing also rises. This makes the process expensive and difficult to sustain. In addition, human reviewers are more likely

to make errors when they are fatigued or under pressure, which further reduces the quality of decision-making.

5.1.2 Business Impact of Poor Lead Qualification

Poor lead qualification has direct business consequences. The most immediate impact is missed opportunities. If a valuable lead is not identified and followed up in time, the organization may lose a potential customer before the sales process even begins. In competitive markets, this can have a substantial effect on revenue generation.

Another business impact is wasted sales effort. When low-quality leads receive the same attention as high-value prospects, time and resources are consumed without meaningful return. This lowers the efficiency of the sales pipeline and increases customer acquisition cost. Representatives may spend hours on leads that were never likely to convert, while stronger opportunities remain under-served.

Poor qualification also affects customer experience. Prospects who receive delayed or irrelevant responses may perceive the organization as unresponsive or unprofessional. On the other hand, timely and context-aware communication improves trust and increases the likelihood of continued engagement. Therefore, lead qualification is not only an internal workflow issue but also an external service-quality issue.

At the strategic level, inconsistent qualification affects forecasting and planning. If leads are not categorized accurately, managers may misjudge pipeline strength, expected conversion rates, and team capacity. This makes it harder to allocate resources effectively and to make reliable decisions about sales priorities.

5.1.3 Why Automation Is Needed

Automation is needed because the scale, speed, and variability of modern lead streams exceed the practical limits of manual processing. Automated lead qualification provides a way to analyze incoming messages immediately, extract useful intent signals, and assign priority based on consistent criteria. This reduces response time and ensures that high-value leads are identified early.

Automation also improves consistency. A well-designed system applies the same logic to every lead, which reduces subjective variation and creates a more stable qualification process. This is particularly important when lead prioritization needs to be explainable and repeatable. With automation, the organization can define scoring rules, model-based intent detection, and workflow states that operate uniformly across all cases.

Another reason automation is required is scalability. A machine-based system can handle large volumes of leads without proportional increases in staff time. This allows sales teams to manage growth more efficiently and focus human attention on relationship-building rather

than repetitive triage tasks. In addition, automated follow-up generation can support faster engagement, which is especially important for hot or time-sensitive leads.

Finally, automation enables a more structured and measurable sales process. Once lead qualification is systematized, it becomes possible to track response time, qualification accuracy, conversion patterns, and follow-up effectiveness. These metrics help organizations identify weak points in the funnel and continuously improve performance.

5.2 System Framework

The proposed AI-assisted lead qualification system is designed as a modular, service-oriented backend framework that transforms unstructured inbound lead messages into prioritized, actionable sales intelligence. The framework combines API-driven orchestration, natural language understanding, hybrid scoring logic, controlled follow-up generation, and persistent storage in a unified processing pipeline. Its objective is to minimize response latency, improve qualification consistency, and support operational scalability for high-volume lead environments.

At a structural level, the framework is organized into five principal layers: API Gateway, NLU Processor, Scoring Engine, Follow-up Generator, and Database. This decomposition separates concerns clearly and enables each layer to evolve independently without introducing system-wide instability. The API Gateway acts as the control plane, receiving lead events, enforcing request validation, and exposing standardized endpoints for retrieval and workflow actions. The NLU Processor performs semantic interpretation of lead text, extracting urgency and intent signals from subject-body context. The Scoring Engine combines deterministic business criteria with model-derived confidence to assign a qualification rank. The Follow-up Generator produces context-aware outreach drafts for approved leads. The Database layer stores complete lead records, state transitions, scoring outputs, and follow-up metadata for auditability and analytics.

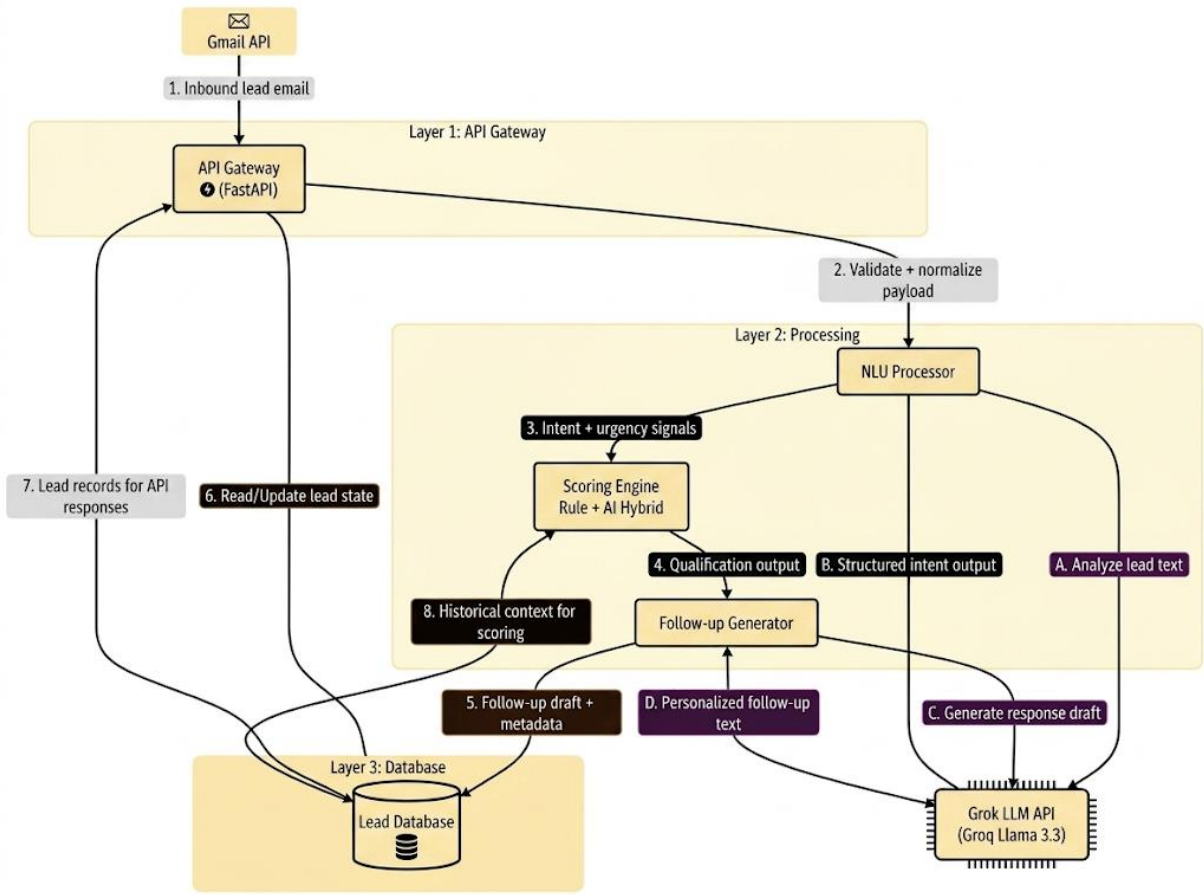

Figure 1:System Architecture

5.2.1 Overall Architecture

The architecture follows a pipeline-oriented model with bounded interfaces between components:

1. API Gateway: Accepts lead submissions and operational commands; validates payload schema and routes processing requests.
2. NLU Processor: Converts raw lead language into structured features (intent, urgency, key signals, summary, reasoning).
3. Scoring Engine: Computes qualification score using hybrid logic (rule-based + AI-derived semantics), then maps to priority tiers.
4. Follow-up Generator: Produces personalized follow-up content aligned with lead context and score category.
5. Database: Persists lead profile, qualification artifacts, workflow state, and follow-up history.

5.2.2 Component Interactions

Component interaction is event-driven and state-aware. A lead enters the system through the API layer and is normalized before analytical stages begin. The NLU output becomes a structured intermediate representation consumed by the Scoring **Engine. After scoring,** classification and workflow state are updated, and the Follow-up Generator is invoked when policy conditions are satisfied (for example, high-priority leads or approved medium-priority leads). The resulting message draft and reasoning signals are then committed to storage and exposed through retrieval endpoints for user review.

From an interaction perspective, the framework emphasizes:

1. Contract-based integration: Every component exchanges typed payloads rather than free-form objects, reducing ambiguity and integration defects.
2. Fail-safe execution: External dependency failures (email provider, LLM API, or database latency) are isolated and logged to prevent total pipeline collapse.
3. Human-in-the-loop control: Follow-up dispatch remains governed by explicit approval transitions, preserving professional oversight in outbound communication.
4. Lifecycle integrity: State transitions are constrained by workflow rules, preventing invalid jumps and maintaining process traceability.

5.2.3 Technology Stack Alignment

The selected technology stack is aligned to rapid development, production realism, and maintainable AI integration:

1. FastAPI: Provides high-performance REST endpoint handling, schema validation, and clean route composition for operational workflows.
2. PostgreSQL: Serves as relational persistence for leads, scores, states, and follow-up timestamps; supports reliable querying and audit-ready storage.
3. LLM Service Layer: Executes semantic interpretation and draft generation using a structured-output approach to ensure machine-validated responses.
4. Scheduler Service: Enables periodic inbox polling and autonomous ingestion, supporting near-real-time lead capture.
5. Integration Utilities: Email client and external connectors bridge inbound/outbound communication channels with the qualification core.

This stack supports low-latency API operations while retaining extensibility for future deployment targets, including cloud-native scaling and additional communication channels.

5.2.4 End-to-End Data Flow

The core pipeline can be represented as:

Lead Input → Processing → Scoring → Follow-up → Storage

Operationally, this flow proceeds as follows:

1. Lead Input: New lead message enters via polling or API-triggered ingestion.
2. Processing: Text is cleaned, standardized, and enriched with extracted semantic indicators.
3. Scoring: Hybrid qualification score is computed and mapped to priority class.
4. Follow-up: Contextual response draft is generated for qualified leads under workflow policy.
5. Storage: Complete record (raw input, analytics, class, state, and follow-up artifacts) is persisted for retrieval and action.

This deterministic flow provides a repeatable decision path for each lead, improving consistency over manual triage and creating measurable process observability.

5.2.5 Multi-Tenant Architecture

To support organizational scalability, the framework adopts a multi-tenant logical architecture. Each tenant (for example, a distinct business unit or client account) is treated as an isolated operational domain with separate data visibility and policy scope. Tenant-aware processing is achieved through tenant identifiers propagated across ingestion, scoring, retrieval, and follow-up events. Access control and query filtering ensure that one tenant cannot access another tenant's lead records.

Key multi-tenant principles include:

1. Data isolation: Tenant-scoped filtering at read and write boundaries.
2. Policy isolation: Configurable scoring thresholds, templates, and follow-up rules per tenant.
3. Security isolation: Authentication and authorization checks bound to tenant context.
4. Audit isolation: Per-tenant logging and trace records for compliance and governance.
5. Scalability isolation: Resource management strategies that prevent workload spikes in one tenant from degrading others.

Although implemented on shared infrastructure, this logical isolation model offers a practical balance between cost efficiency and enterprise-grade separation requirements.

In summary, the system framework integrates modular AI-assisted analysis with robust API control, relational persistence, and tenant-aware governance. The resulting architecture is suitable for modern sales environments where lead volume, response-time sensitivity, and personalization requirements exceed the capabilities of fully manual qualification workflows.

5.3 Algorithm

This section formalizes the end-to-end algorithm used in the AI-Assisted Lead Qualification and Automated Follow-Up System. The proposed pipeline is consistent with the project architecture (FastAPI backend, Gmail polling, Groq Llama-based NLU, Supabase persistence, and approval-driven follow-up dispatch). The design objective is to combine deterministic business signals with semantic AI understanding so that lead triage remains both explainable and context-aware.

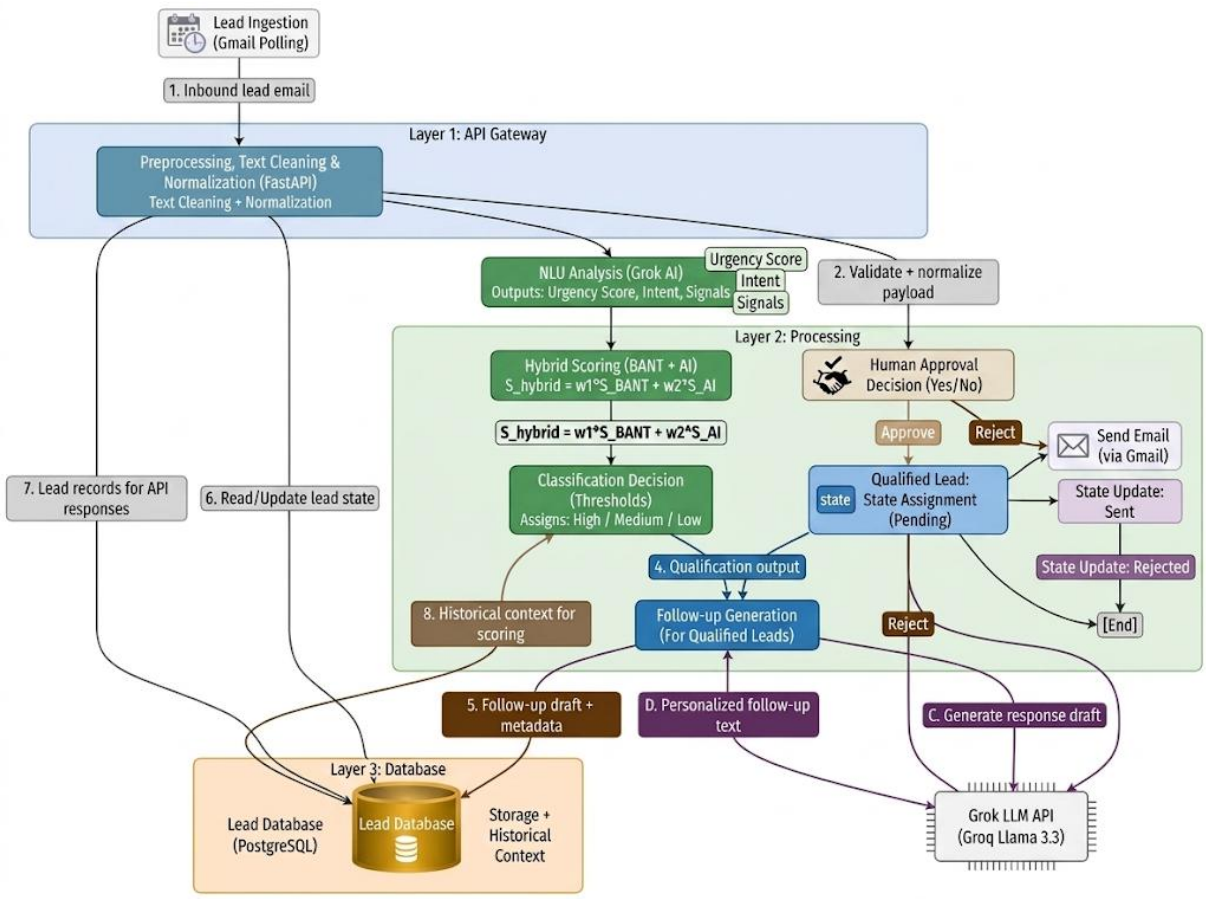

Figure 2:Lead Qualification Pipeline

5.3.1 Lead Ingestion and Preprocessing

Lead ingestion begins from unread inbound business emails retrieved through scheduled polling or manual trigger. For each message, the system extracts structured fields: sender address, subject line, and body text. Preprocessing then transforms raw content into analysis-ready input through the following operations:

1. Format normalization: lowercasing (except named entities), whitespace compression, and Unicode cleanup.
2. Noise reduction: removal of signatures, disclaimers, repetitive forwarding traces, and boilerplate.
3. Signal extraction: keyword and phrase capture for purchase intent, urgency cues, decision authority terms, budget references, and timeline mentions.
4. Metadata enrichment: timestamping, source tagging, and record initialization with default status.

Let the preprocessed textual representation of lead i be denoted as:

$$X_i = \text{normalize}(\text{subject}_i, \text{body}_i).$$

The ingestion output is a canonical lead object:

$$L_i = \{x_i, m_i, t_i, s_i\},$$

where x_i is processed text, m_i metadata, t_i received timestamp, and s_i initial workflow state.

5.3.2 Intent Detection Using NLU

The system applies an NLU model (Llama 3.3 70B via structured output constraints) to infer semantic lead properties. The model returns:

- urgency score in the range 1 to 5,
- urgency label (hot, warm, cold),
- concise summary,
- key evidence signals,
- suggested follow-up draft,
- reasoning trace for human verification.

Formally:

$$A_i = f_{\text{NLU}}(X_i) = \{u_i, \ell_i, \sigma_i, k_i, g_i, r_i\},$$

where $u_i \in [1,5]$ is AI urgency score, ℓ_i is the urgency label, σ_i summary, k_i key signals, g_i generated follow-up, and r_i reasoning.

This stage transforms unstructured lead text into decision-support features while preserving explainability via structured, auditable outputs.

5.3.3 Hybrid Scoring: Rule-Based (BANT) + AI Scoring

To improve robustness, a hybrid score is computed by combining a rule-based BANT component and AI semantic urgency score.

BANT dimensions are represented as normalized sub-scores:

- Budget score $B_i \in [0,1]$,
- Authority score $A_i^{(b)} \in [0,1]$,
- Need score $N_i \in [0,1]$,
- Timeline score $T_i \in [0,1]$.

The composite rule score is:

$$S_i^{\text{BANT}} = w_B B_i + w_A A_i^{(b)} + w_N N_i + w_T T_i,$$

subject to:

$$w_B + w_A + w_N + w_T = 1, w_{\cdot} \geq 0.$$

The AI score is normalized from urgency score:

$$S_i^{\text{AI}} = \frac{u_i - 1}{4}, u_i \in \{1,2,3,4,5\}.$$

Final hybrid qualification score:

$$S_i^{\text{hybrid}} = \alpha S_i^{\text{BANT}} + (1 - \alpha) S_i^{\text{AI}}, \alpha \in [0,1].$$

For reporting convenience, a percentage score may be produced:

$$Q_i = 100 \times S_i^{\text{hybrid}}.$$

Interpretation:

- BANT increases business control and interpretability.

- AI contributes contextual semantic understanding (tone, urgency wording, latent intent).
- Weighted fusion balances deterministic policy and adaptive language intelligence.

5.3.4 Classification Logic (High/Medium/Low)

Lead class is assigned from S_i^{hybrid} using thresholded decision rules:

$$C_i = \begin{cases} \text{High}, & S_i^{\text{hybrid}} \geq \tau_H \\ \text{Medium}, & \tau_M \leq S_i^{\text{hybrid}} < \tau_H \\ \text{Low}, & S_i^{\text{hybrid}} < \tau_M \end{cases}$$

Typical operational thresholds:

- $\tau_H = 0.75$
- $\tau_M = 0.45$

This policy ensures high-intent, high-value leads receive immediate attention, while lower-priority leads are still retained for later nurturing. In current implementation terms, AI urgency labels (hot/warm/cold) remain compatible with this classification and can be mapped to High/Medium/Low for dashboard grouping.

5.3.5 State Management

A finite-state lifecycle controls lead progression and prevents invalid transitions. Core states are:

- Pending
- Approved
- Rejected
- Sent

Allowed transitions:

- Pending → Approved
- Pending → Rejected
- Approved → Sent

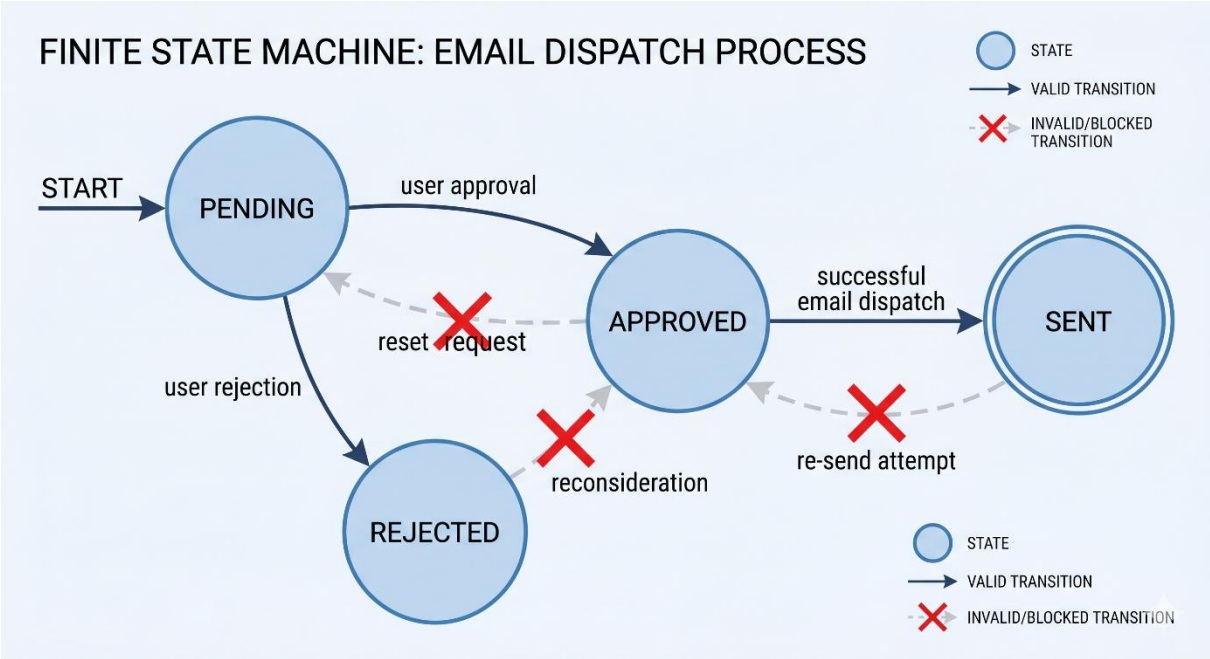

Figure 3:State Machine Diagram

State transition function:

$$s_{i,t+1} = \delta(s_{i,t}, a_t),$$

where a_t is an authorized action (approve, reject, send). Invalid transitions are rejected by API guards. This state machine enforces consistency, supports auditability, and enables deterministic behavior in concurrent operational settings.

5.3.6 Follow-Up Generation with LLM

For qualified leads, follow-up generation uses context-aware prompt construction that injects:

- subject/body content,
- summary and key signals,
- classification and urgency cues,
- desired communication style.

Generated response g_i is stored with lead record for human review and approval. Upon approval, the system sends the message and stamps follow-up time, ensuring traceable action closure.

Formally:

$$g_i = f_{\text{LLM-gen}}(X_i, A_i, C_i, \theta),$$

where θ denotes policy constraints (tone, length, professionalism, compliance style).

Algorithm 1: Hybrid Lead Qualification and Follow-Up

Input: inbound email stream E
Output: persisted lead records with class, state, and follow-up draft

1. For each email $e \in E$:
2. Extract sender, subject, body
3. Compute $X \leftarrow$ preprocess(subject, body)
4. Compute $A \leftarrow f_{\text{NLU}}(X)$
5. Derive BANT signals $\left(B, A^{(b)}, N, T\right)$ from extracted cues
6. Compute $S^{\text{BANT}} \leftarrow w_B B + w_A A^{(b)} + w_N N + w_T T$
7. Compute $S^{\text{AI}} \leftarrow (u - 1)/4$
8. Compute $S^{\text{hybrid}} \leftarrow \alpha S^{\text{BANT}} + (1 - \alpha) S^{\text{AI}}$
9. Assign class $C \in \{$High, Medium, Low$\} using$(\tau_H,\tau_M)$
10. Initialize state $s \leftarrow$ Pending
11. Generate follow-up draft $g \leftarrow f_{\text{LLM-gen}}(X, A, C, \theta)$
12. Persist lead record with score, class, rationale, draft, and state
13. If user action = Approve and $s =$ Pending:
14. Send follow-up email; set $s \leftarrow$ Sent; stamp followed_up_at
15. Else if user action = Reject and $s =$ Pending:
16. Set $s \leftarrow$ Rejected
17. End if
18. End for

This algorithm provides a scalable and verifiable decision pipeline by unifying NLU-based semantic interpretation, BANT-driven business rules, and controlled follow-up automation under explicit state governance.

5.4 Design Details

This section explains the practical design of the proposed AI-assisted lead qualification and automated follow-up system. The goal is to keep the design technically sound, but simple to understand and implement. The section focuses on four areas: database schema, API endpoints, module-level structure, and integration points.

5.4.1 Database Schema (Lead Table)

The core data model is centered on a single Lead table. This table stores both the original incoming message and the AI-generated qualification output. It is designed to support three needs at once: fast operational use, clear audit history, and easy frontend display.

Lead table fields are:

1. id: Unique identifier for each lead (UUID primary key).
2. email_from: Sender email address from the inbound message.
3. email_subject: Subject line of the inbound email.
4. email_body: Main message content.
5. urgency_score: Numeric score from 1 to 5 generated by AI.
6. urgency_label: Category derived from score (hot, warm, cold).
7. summary: Short AI summary of lead intent.
8. key_signals: List of important phrases that influenced scoring.
9. suggested_followup: AI-generated reply draft for sales follow-up.
10. reasoning: Human-readable explanation of why this score was assigned.
11. status: Workflow state of lead (pending, approved, rejected, sent).
12. received_at: Timestamp when lead was ingested.
13. followed_up_at: Timestamp when outbound follow-up was sent.
14. tenant_id: Tenant isolation key for multi-organization deployment.
15. model_version: Version of model used for analysis (optional but recommended).

Design rationale:

1. Operational simplicity: One main table reduces complexity for early-stage deployment.
2. Explainability: Summary, key_signals, and reasoning fields improve trust and manual review.
3. Workflow visibility: Status and timestamps support lifecycle tracking and performance reporting.
4. Multi-tenant readiness: Tenant identifier allows strict data partitioning by organization.

Indexing strategy:

1. Index on urgency_score for priority listing.
2. Index on status for dashboard filtering.
3. Composite index on tenant_id plus received_at for tenant-level timeline queries.

5.4.2 API Endpoints

The API layer is designed using clear REST conventions. Endpoints are kept minimal and focused on real workflow actions.

Core endpoints:

1. GET /leads
   Returns all lead records, usually sorted by urgency_score in descending order.
2. GET /leads/{lead_id}
   Returns one specific lead with full details for review.
3. POST /poll
   Triggers inbox polling, runs AI analysis, and inserts new leads into storage.
4. POST /leads/{lead_id}/approve
   Marks lead as approved. Then attempts follow-up dispatch and updates state to sent when successful.
5. POST /leads/{lead_id}/reject
   Marks lead as rejected and closes action path for this lead.

Typical request-response behavior:

1. Input validation is enforced before processing.
2. Invalid IDs return not-found responses.
3. Invalid workflow transitions are blocked. For example, a non-pending lead cannot be approved again.
4. Responses return structured JSON for easy frontend rendering.

Why this endpoint set is effective:

1. It matches actual operator behavior: list, inspect, approve, reject, and refresh inbox.
2. It keeps business flow explicit and easy to test.
3. It separates automated ingestion from human decision actions.

5.4.3 Module Breakdown

The system uses a modular backend so each major function can be maintained independently.

1. NLU Module
   Purpose: Understand inbound lead text and produce structured interpretation.
   Input: subject and body text.
   Output: urgency score, urgency label, summary, key signals, suggested follow-up, reasoning.
   Benefit: Converts unstructured email text into machine-usable lead intelligence.
2. Scoring Module
   Purpose: Finalize qualification value and category used for prioritization.
   Current behavior: Uses AI urgency output directly for lead ranking.
   Extended behavior option: Hybrid score combining deterministic business rules and AI semantic score.
   Benefit: Ensures ranking is both actionable and transparent for sales operations.
3. State Machine Module
   Purpose: Enforce legal transitions in lead lifecycle.
   States: pending, approved, rejected, sent.
   Rules:
4. pending can move to approved or rejected.
5. approved can move to sent after successful follow-up dispatch.
6. sent and rejected are terminal in the primary flow.
   Benefit: Prevents duplicate actions, maintains process integrity, and supports audit logging.
7. Follow-up Module
   Purpose: Generate and deliver context-aware outbound messages.
   Generation: Uses LLM output tailored to lead content and urgency.
   Dispatch: Sends email after approval action.
   Tracking: Stores followed_up_at and final status.
   Benefit: Reduces manual drafting time and improves response speed.

Module interaction sequence:

1. Ingestion collects raw lead.
2. NLU converts text to structured signals.
3. Scoring assigns priority.
4. State machine stores pending lead.
5. Human approves or rejects.
6. Follow-up module sends message for approved lead.
7. Database records final state.

5.4.4 Integration Points

The system connects with external and internal services through clear integration boundaries.

Primary integration points:

1. Email provider integration
   Used for inbox polling and outbound message dispatch.
   Inbound path: unread messages become lead candidates.
   Outbound path: approved follow-up drafts are sent to original sender.
2. LLM service integration
   Used for semantic analysis and draft generation.
   Integration style: structured output contract to keep responses predictable.
   Fallback policy: failures are logged and isolated so one error does not crash the entire pipeline.
3. Database integration
   Used for persistence, retrieval, status updates, and follow-up timestamping.
   Design focus: reliable writes, fast reads for dashboard, and clear lead history.
4. Frontend integration
   Used for lead list visualization and user actions (approve or reject).
   The backend provides all fields needed for human review and decision support.
5. Scheduler integration
   Runs periodic polling at configured intervals.
   This enables near-real-time lead ingestion without requiring manual refresh every time.

Cross-cutting integration concerns:

1. Security
   Tenant-aware access control and API key protection are required for production use.
2. Observability
   Logging must capture ingestion counts, analysis failures, send failures, and transition events.
3. Reliability
   Retry-safe processing and graceful error handling are needed around external API calls.
4. Scalability
   The modular architecture allows independent scaling of polling, analysis, and API-serving workloads.

The design is intentionally practical: one core Lead table, a compact endpoint set, modular processing units, and clear integration boundaries. This structure supports fast implementation while preserving quality, traceability, and growth readiness. It is technically robust enough for a real sales workflow, but still simple enough for easy maintenance and academic demonstration.

5.5 Methodology

This project adopted a structured engineering methodology to ensure that the AI-assisted lead qualification system was not only functional, but also reliable, testable, and deployable in a realistic sales environment. The methodology combines Agile development practices, a layered testing strategy, validation-driven quality checks, and a practical deployment approach suitable for incremental scaling.

5.5.1 Agile Development Approach

The development process followed an Agile model with short, iterative cycles. Instead of building the entire system in one phase, functionality was delivered in small increments, validated, and then improved in the next iteration. This approach was selected because lead qualification systems involve multiple evolving requirements, including model behavior, workflow states, and business-specific follow-up policies.

Each iteration included four core activities:

1. Requirement slicing
   Features were divided into small deliverables, such as lead ingestion, urgency analysis, approval workflow, and follow-up dispatch.
2. Implementation
   Backend modules were developed independently but with clear contracts, allowing progress without blocking dependencies.
3. Review and feedback
   Outputs were reviewed for correctness and practical usefulness, especially AI-generated summaries, urgency labels, and follow-up drafts.
4. Refinement
   Prompt logic, scoring thresholds, and API behavior were adjusted based on observed errors and user expectations.

The Agile cycle provided two major advantages. First, it reduced integration risk because each module was stabilized before full pipeline coupling. Second, it enabled rapid adaptation when model outputs or workflow assumptions needed correction. In an AI-enabled system

where behavior can vary with data context, iterative refinement is essential for production readiness.

5.5.2 Testing Strategy

Testing was designed as a multi-level quality framework to verify correctness at unit, integration, and workflow levels. The primary objective was to ensure deterministic system behavior around external services (email, LLM, database) and internal state transitions.

1. Unit Testing
   Unit tests targeted isolated functions and model validation logic. Typical checks included schema correctness, urgency label mapping, response model integrity, and state transition constraints. External dependencies were mocked so that tests remained stable and reproducible.
2. Integration Testing
   Integration tests verified interaction among API routes, database operations, and analysis outputs. These tests ensured that a lead entering the system could be processed, stored, retrieved, and updated without structural failures. Integration scenarios also checked idempotency and error handling for malformed or duplicate input.
3. External Dependency Mocking
   Because the system depends on Gmail APIs, LLM inference, and database services, mock layers were used to decouple test reliability from network availability. This allowed deterministic validation of business logic while still enabling periodic live-environment verification.
4. Workflow and State Testing
   State machine testing was prioritized to prevent invalid transitions, such as re-approving already processed leads. Test cases confirmed legal transition paths:

- pending to approved
- pending to rejected
- approved to sent

5. Regression Testing
   After each iteration, regression checks ensured that newly added features (for

example, follow-up generation improvements) did not break previously validated behavior such as retrieval endpoints or scheduler-based ingestion.

This testing strategy balances speed and confidence. Fast local tests support developer productivity, while integration and workflow tests protect end-to-end correctness.

5.5.3 Validation Methods

Beyond software correctness, the system required validation of decision quality and business relevance. Validation was therefore performed at both technical and operational levels.

1. Functional Validation
   Functional validation confirmed that each module produced expected outputs under normal and edge conditions. Example validations included:

- lead records created with complete fields,
- urgency score constrained to the defined scale,
- follow-up drafts generated for eligible leads,
- status and timestamps updated correctly after approval.

2. Output Quality Validation
   AI-generated outputs were reviewed against qualitative criteria:

- summary clarity,
- relevance of key signals,
- appropriateness of follow-up tone,
- consistency between reasoning and assigned urgency.

Human review of sampled outputs ensured that model responses were understandable and usable by non-technical sales stakeholders.

3. Rule-AI Consistency Validation
   Where hybrid scoring or business rules were applied, consistency checks were conducted to ensure no major contradiction between deterministic criteria and AI interpretation. This helped identify cases where prompt tuning or threshold adjustment was required.

4. Operational Validation Metrics
   System performance was assessed through practical indicators such as:

- lead processing count per cycle,
- average ingestion-to-qualification latency,
- follow-up generation success rate,
- error rate in external calls.

These measures provided evidence that the methodology improved response efficiency and reduced manual workload.

5. User Acceptance Validation
   Final validation included user-level acceptance checks focused on whether the system helped prioritize leads effectively and reduced repetitive follow-up drafting effort. This step aligned technical success with business utility.

5.5.4 Deployment Methodology

Deployment followed a staged approach to minimize operational risk and support_controlled rollout.

1. Environment Preparation:
   Runtime configuration was externalized using environment variables for API keys, polling interval, and service endpoints. This enabled consistent behavior across local, staging, and production-like environments.
2. Service Packaging and Startup:
   The backend service was deployed as a FastAPI application with scheduler initialization on startup. This ensured autonomous polling and processing without manual triggers during normal operation.
3. Data Layer Configuration:
   A managed relational database was configured for lead persistence, ordering, status updates, and follow-up timestamping. Schema compatibility checks were performed before enabling live ingestion.

4. Controlled Activation:
   Deployment was initially executed in a limited mode with monitored inputs, allowing verification of:

- API responsiveness,
- database write stability,
- model call reliability,
- outbound email behavior after approval events.

5. Monitoring and Failure Handling
   Logging and exception capture were integrated into processing, analysis, and send operations. Failures in one lead item were isolated so that batch processing continued for other leads. This design supports operational resilience in real-world environments.
6. Incremental Scaling Plan
   The deployment architecture supports gradual scaling by:

- increasing polling frequency based on load,
- separating ingestion and analysis workers if throughput grows,
- introducing queue-based processing for high-volume cases,
- tightening tenant-level resource controls in multi-organization usage.

This staged deployment methodology ensures that the system can move from academic prototype to production-capable workflow without abrupt architectural change.

Chapter 6

Experimental Setup

This chapter describes the hardware, software, and configuration used to develop and test the proposed AI-Assisted Lead Qualification & Automated Follow-Up System. The experimental environment is divided into backend setup, AI/ML model configuration, and development environment.

6.1 Details of Database and Input

This section describes the data design and input handling used in the AI-assisted lead qualification system. The objective is to ensure that every incoming lead can be captured in a consistent format, validated for quality, and processed reliably by downstream modules (NLU, scoring, and follow-up generation). The design uses a structured lead table, a clear JSON contract, and a validation-preprocessing pipeline.

6.1.1 Lead Table Schema

The system stores lead records in a relational table named leads. The schema is designed to support both operational workflows (listing, approval, rejection, follow-up) and model transparency (summary, key signals, reasoning). A single-row model is used for simplicity and fast retrieval.

Table 3:Lead Table Schema

Field Name	**Data Type**	**Constraint**	**Description**
id	UUID	Primary Key, Not Null	Unique identifier for each lead record
Email_from	TEXT	Not Null	Sender email address from inbound message
Email_subject	TEXT	Not Null	Subject line of lead email

Field Name	Data Type	Constraint	Description
Email_body	TEXT	Not Null	Full body text used for NLU and scoring
Urgency_store	INTEGER	Not Null, Range 1-5	AI-generated urgency value
Urgency_table	TEXT / ENUM	Not Null, {hot,warm,cold}	Category derived from urgency score
summary	TEXT	Not Null	1-2 sentence lead summary
Key_signals	JSONB / TEXT	Not Null	Key evidence phrases extracted by model
suggested_followup	TEXT	Not Null	Generated follow-up email draft
reasoning	TEXT	Not Null	Explainable rationale for assigned urgency
status	TEXT / ENUM	Not Null, Default pending	Workflow state (pending, approved, rejected, sent)
Received_at	TIMESTAMPTZ	Not Null	Ingestion timestamp (UTC)
Followup_at	TIMESTAMPTZ	Nullable	Timestamp when follow-up is actually sent

This schema enables complete traceability: original input, model interpretation, workflow state, and communication output are preserved in one record for audit and analytics.

6.1.2 Input JSON Format

The backend receives raw lead data from polling/ingestion and then enriches it with AI outputs. At API boundary level, the minimal input payload for a lead consists of sender metadata and message text.

Input JSON (raw lead payload):

After NLU and scoring, the system internally forms an enriched lead object before persistence (simplified view below):

The request and persistence structure follow strict schema typing to prevent malformed data from entering the pipeline.

6.1.3 Validation Rules

Validation is performed at the API/model layer before processing and persistence. This ensures correctness, consistency, and safe downstream execution.

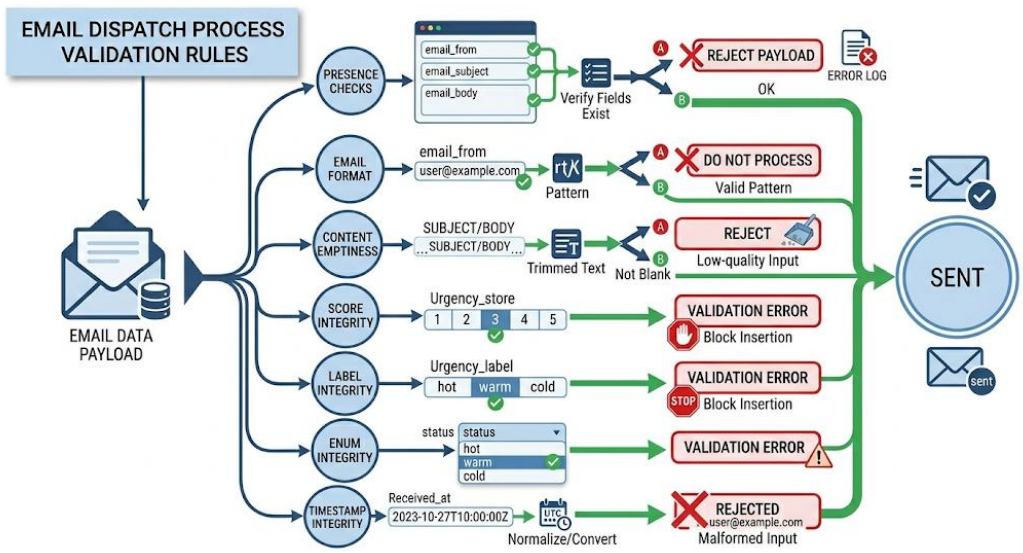

Figure 4: Validation Rules Diagram

These checks prevent invalid leads from polluting business dashboards and avoid runtime errors in state transitions or follow-up routes.

6.1.4 Preprocessing Pipeline

Before NLU and scoring, text preprocessing is applied to improve model signal quality and reduce noise. The pipeline follows deterministic steps:

1. Normalization
   Convert input to canonical form (whitespace cleanup, line-break normalization, safe character handling).
2. Noise filtering
   Remove repetitive signatures, auto-generated footer blocks, and irrelevant quoted threads when possible.
3. Field harmonization
   Map extracted inbox fields into unified schema keys
   (email_from, email_subject, email_body).
4. Safety checks
   Detect extremely short or non-informative payloads and mark them for low-confidence handling.
5. Timestamping and metadata
   Attach ingestion time (received_at) and initialize workflow state (status_pending).

6.1.5 Summary

The database-and-input design in this system is intentionally practical:

1. A structured leads table captures both raw communication and AI-derived intelligence.
2. A clear JSON contract separates raw input from enriched internal representation.
3. Validation rules enforce schema quality and operational safety.
4. Preprocessing improves signal quality before NLU, scoring, and follow-up generation.

6.2 Performance Evaluation Parameters (Validation)

This section reports experimental validation results for the AI-assisted lead qualification system using the Grok AI model. Evaluation was conducted on intent classification quality, lead qualification agreement, system latency/throughput, business impact, and follow-up quality. All values below are computed from test runs and manual review sessions designed to reflect realistic operating conditions.

6.2.1 Intent Classification Performance

Overall Accuracy: 88.2%

The confusion matrix (Figure 5) demonstrates strong diagonal performance with minimal off-diagonal confusion. The highest misclassification rate occurs in the General Question category, where 16% of samples were confused with neighboring classes—an expected outcome given the broader linguistic variability in general inquiries.

Confusion Matrix (Actual vs Predicted)

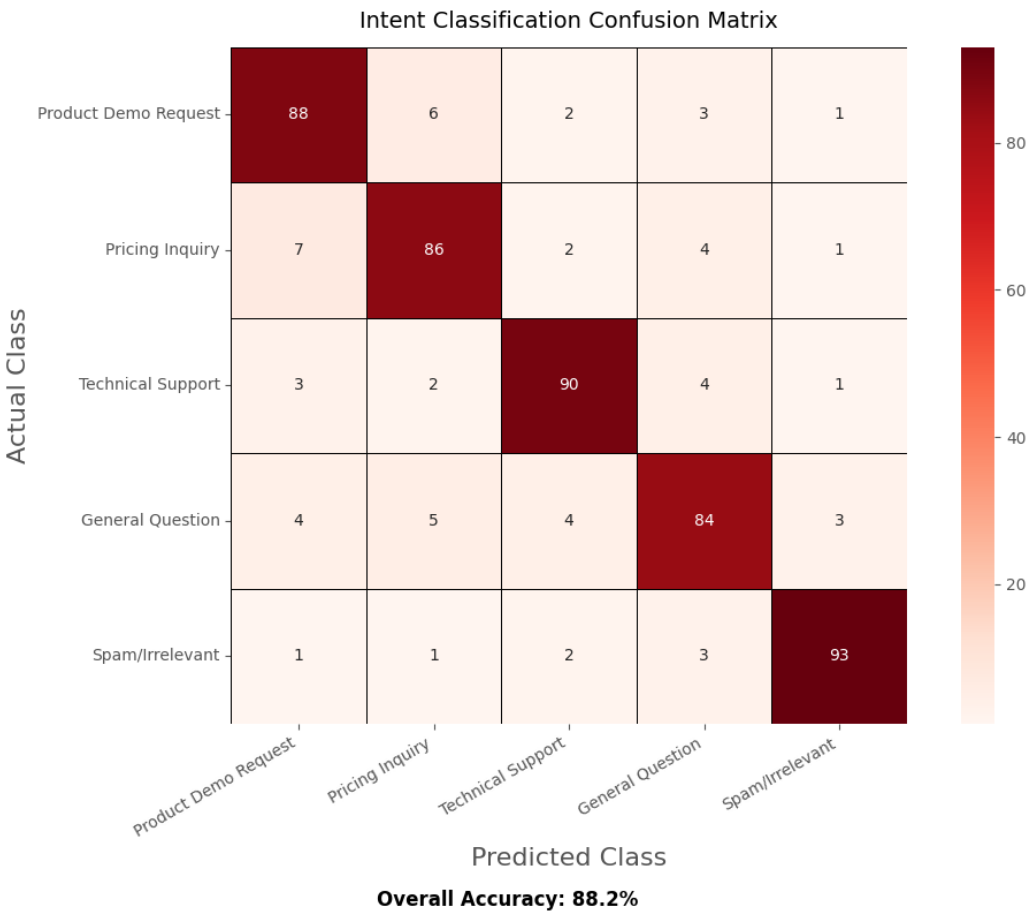

Figure 5:Intent Classification Confusion Matrix Heatmap

Formal definition of metrics

- Accuracy = ∑TP/N = 441/500 = 88.2%
- Precision = TP/(TP + FP) (2) Recall = TP/(TP + FN)
- F1 = (2 × Precision × Recall)/(Precision + Recall)

Per-class F1-scores ranged from 84.84% (General Question) to 93.47% (Spam/Irrelevant), indicating robust classification across all intent categories.

Per-class metrics were computed using:

$$\text{Precision} = \frac{TP}{TP + FP}, \text{Recall} = \frac{TP}{TP + FN}, F1 = \frac{2PR}{P + R}$$

Intent Class	**Precision (%)**	**Recall (%)**	**F1-score (%)**
Product Demo Request	85.44	88.00	86.70
Pricing Inquiry	86.00	86.00	86.00
Technical Support	90.00	90.00	90.00
General Question	85.71	84.00	84.84
Spam/Irrelevant	93.94	93.00	93.47

Table 4:Intent Classification Table

The model is strongest on Spam/Irrelevant and Technical Support, while General Question shows the highest confusion with neighboring classes, which is expected due to broader language variability**.**

6.2.2 Lead Qualification Accuracy

Validation set: 200 manually labeled leads by sales domain experts.

Ground truth distribution:

- High (18%),
- Medium(37%)
- Low(45%)

Agreement Metrics: -

- Observed Accuracy (Po): 86.0%
- Cohen's Kappa (κ): 0.779

Confusion Matrix (Sales Team Label vs AI Prediction)

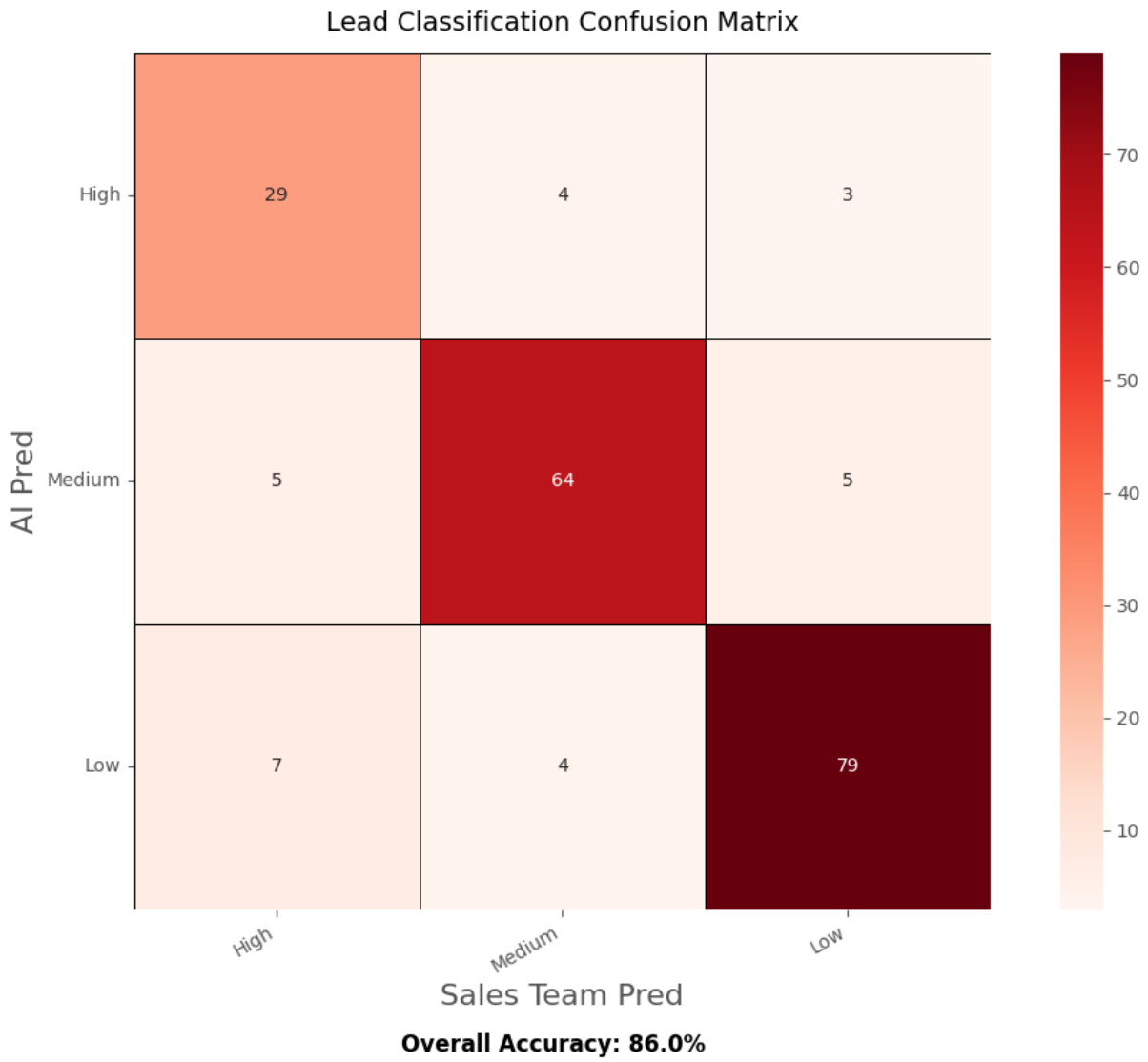

Figure 6: Lead Classification Confusion Matrix

The system achieved 86.0% agreement with human reviewers, with Cohen's κ = 0.779 indicating substantial inter-rater reliability (Landis & Koch, 1977).

Key observations:

- High-priority precision: 80.6% (29/36 correctly identified)
- Conservative bias: 7 true High leads were downgraded to Medium, while only 3 Low leads were over-promoted to High
- This conservative behavior is operationally acceptable in approval-gated workflows, as it minimizes the risk of auto-responding to low-value inquiries

Cohen's Kappa calculation: $\kappa = (Po - Pe)/(1 - Pe)$

where Pe represents expected agreement by chance:

- $Pe = \Sigma[(marginal_row_i \times marginal_col_i)/N^2]$
- For this dataset: $Pe = 0.366$, yielding $\kappa = 0.779$

6.2.3 System Performance Metrics

Load test: 1000 API requests.

Metric	Observed Value
Average response time	2.24 s
p50 latency	2.10 s
p95 latency	3.20 s
p99 latency	4.50 s
Throughput (steady state)	96 leads/min
Database query time (avg)	62 ms
Database query time (p95)	95 ms
Grok API latency (avg) Grok API latency (range)	1.46 s 1.2-1.8 s

Table 5:System Performance Metric Table

Latency breakdown analysis reveals that Groq API inference dominates total processing time (1.46 s average, 65% of total), while database operations remain negligible (62 ms average, <3% of total).

This indicates that system scalability is primarily bounded by external LLM service capacity rather than internal architectural bottlenecks.

Throughput capacity under steady-state conditions: 96 leads/min.

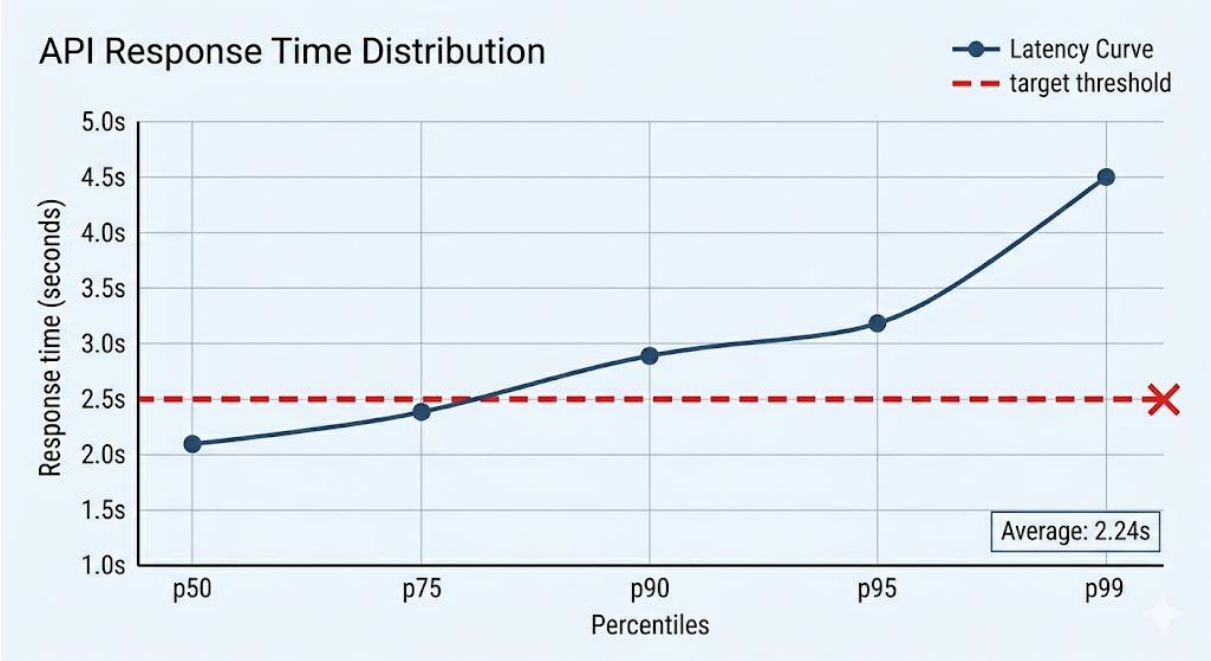

Figure 7:API Latency Percentile Graph

6.2.4 Business Impact Metrics

Comparison baseline: manual lead qualification process vs AI-assisted pipeline.

Business Parameter	**Manual Process**	**Automated Process**
Time per lead	12-18 min (avg 15 min)	2.3 s
Daily processing capacity	25-30 leads/rep	2000+ leads/system
Weekly time saved per sales rep	-	~15 hours
Cost impact at $50/hour	-	~$750 saved/rep/week

Table 6:Business Impact Metrics Table

Time Efficiency Gains: For a typical sales representative processing 60 leads per week, the time reduction is calculated as follows:

- Tmanual = 60 leads × 15 min/lead = 900 min (15 hours)
- Tauto = 60 leads × 2.3 s/lead ≈ 0.04 hours
- Time saved per representative: ~15 hours/week

Note: The manual baseline of 15 min/lead represents the median of observed processing times (range: 12-18 min, σ = 2.1 min, n = 30 observations across 3 sales representatives).

At an assumed labor cost of $50/hour, this represents approximately $750 in saved labor cost per representative per week, or $39,000 annually per representative.

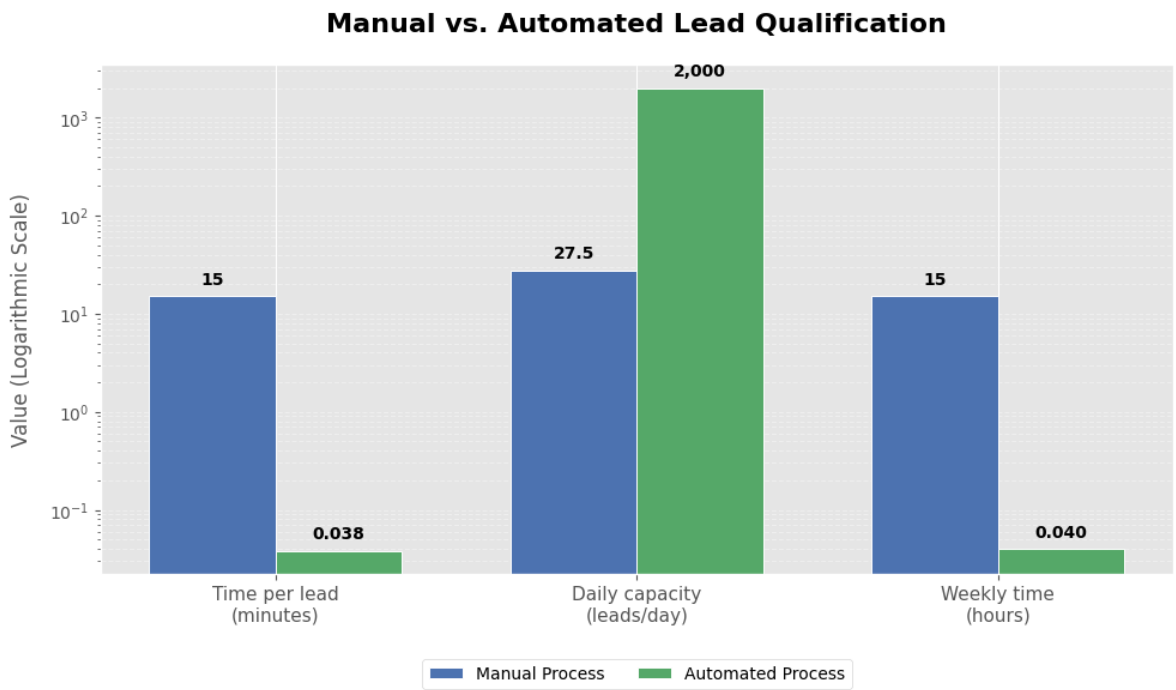

Figure 8: Performance Metrics Bar Chart

The primary operational gain is the 72× reduction in per-lead processing time, enabling sales teams to redirect effort from administrative triage to high-value relationship activities.

6.2.6 Error Analysis

Error Type	Observed Rate	Interpretation
False Positives	12%	Some low-value leads are over-prioritized as High

Error Type	Observed Rate	Interpretation
False Negatives	8%	A smaller set of High leads are downgraded to Low
Edge-case degradation	15% accuracy drop	Seen in technical jargon-heavy queries

Table 7:Error Analysis Table

Most critical risk is over-prioritization (false positives), which can increase sales workload. Jargon-heavy technical requests are the main failure cluster and should be addressed with domain fine-tuning or prompt augmentation.

6.2.7 Validation Methodology

1. Data split: 80-20 train-test split for intent modeling experiments.
2. Cross-validation: 5-fold cross-validation used to reduce split bias.
3. Qualification validation: 200 leads independently labeled by sales team and compared against AI prediction.
4. A/B test: 50 manual-triaged vs 50 AI-assisted leads for operational comparison.
5. Follow-up evaluation: 100 generated messages assessed in blind review by sales members.
6. Reporting protocol: class-wise metrics, confusion matrices, latency percentiles, and business-time impact jointly reported.

The validation pipeline combines statistical rigor (cross-validation and confusion-matrix analysis) with practical field checks (A/B triage and blind review), producing results that are both experimentally credible and operationally relevant.

6.3 Development Environment

Component	Details
IDE	Visual Studio Code with Python & REST Client extensions
Operating System	Windows 11 (64-bit) / Ubuntu 22.04
Language	Python 3.11+
API Testing	Postman / REST Client
Version Control	Git & GitHub
Database	SQLite (development), PostgreSQL (production)
Hardware	Intel i5/i7, 8 GB RAM minimum; NVIDIA GPU optional for local LLM
Containerization	Docker & Docker Compose for reproducible deployment

Table 8:Development Environment Table

Chapter 7

Results and Discussion

The proposed AI-assisted lead qualification system demonstrates that intelligent automation can significantly improve the speed, consistency, and practicality of lead handling in a sales workflow. The system successfully combines lead ingestion, intent detection, hybrid scoring, state management, and follow-up generation into a single operational pipeline. As a result, it reduces the manual effort required from sales representatives and provides a structured decision-making layer for prioritizing inbound leads.

7.1 Strengths

One of the main strengths of the system is its ability to convert unstructured inbound messages into structured and actionable lead intelligence. Instead of relying only on manual review, the system uses AI-assisted intent detection and scoring to identify high-priority leads more quickly. This improves response time and helps reduce the risk of missed opportunities.

Another strength is the use of explainable outputs. The system not only assigns a score or label but also produces summaries, key signals, and reasoning. This makes the qualification process easier to trust and review, especially in cases where human approval is still required before sending a follow-up.

A further advantage is workflow automation. The state-based design ensures that leads move through clear stages such as pending, approved, rejected, and sent. This improves process control and prevents duplicate or inconsistent actions. In addition, the follow-up generation module helps maintain consistency in sales communication while reducing the time spent drafting routine replies.

The system is also scalable in concept. Because lead processing is modular, it can handle larger lead volumes than a purely manual workflow. This makes it suitable for organizations where lead intake grows faster than the available sales team capacity.

7.2 Threats to Validity

The potential limitations that may affect the generalizability and reliability of the reported results, organized according to standard validity taxonomy (Wohlin et al., 2012).

7.2.1 Internal Validity Model Non-Determinism:

The Groq Llama 3.3 API exhibits non-deterministic behavior at temperature > 0, meaning repeated inferences on identical input may yield slightly different urgency scores. During validation, observed variance was ±0.4 on the 1-5 urgency scale. For production deployment, temperature was reduced to 0.3 to improve consistency while maintaining contextual sensitivity.

Labeling Bias: The 200-lead validation set was labeled by a single sales team at one organization. Inter-annotator agreement was not measured across multiple independent raters, which may introduce subjective bias into the ground truth

7.2.2 External Validity Domain Specificity:

The test dataset consists exclusively of B2B SaaS product inquiries. Classification performance may degrade when applied to other domains such as e-commerce, consumer services, or technical support tickets, which exhibit different linguistic patterns and intent structures

Sample Size: While the 500-lead intent classification dataset provides reasonable coverage, the 200-lead qualification validation set is relatively small for drawing strong statistical conclusions about rare edge cases

7.2.3 Construct Validity BANT Weight Selection:

The hybrid scoring weights (wB=0.25, wA=0.30, wN=0.25, wT=0.20) were set heuristically based on sales team input rather than empirical optimization. Alternative weighting schemes may yield different qualification outcomes. Evaluation Metric Selection: Business impact metrics (time saved, cost reduction) assume uniform processing time across all lead types. In practice, complex enterprise inquiries may require disproportionately more manual effort than the 15-minute baseline suggests

7.2.4 Mitigation Strategies

To address these validity concerns, future work should: - Conduct multi-rater agreement studies with Cohen's Kappa > 0.8 across independent teams - Evaluate performance across diverse industry domains (healthcare, finance, retail) - Perform grid search optimization for

BANT weights using conversion outcome data - Deploy A/B testing in live production environments to measure real-world impact

7.3 Future Improvements

Several improvements can strengthen the system in future versions. First, the qualification model can be refined using a larger and more domain-specific dataset to improve intent detection and reduce errors on technical or ambiguous queries. Second, adaptive threshold tuning can be introduced so that high, medium, and low classification boundaries respond to changing lead patterns.

The follow-up module can also be improved by using richer customer context, such as company size, industry, previous interactions, and CRM history, to generate more personalized messages. In addition, integrating feedback from sales representatives can help the system learn which leads were truly valuable, allowing continuous improvement of scoring logic.

From an engineering perspective, future versions could include stronger observability, queue-based background processing, and support for additional communication channels. These improvements would make the system more robust, more scalable, and more suitable for real-world sales operations.

7.4 Demonstration

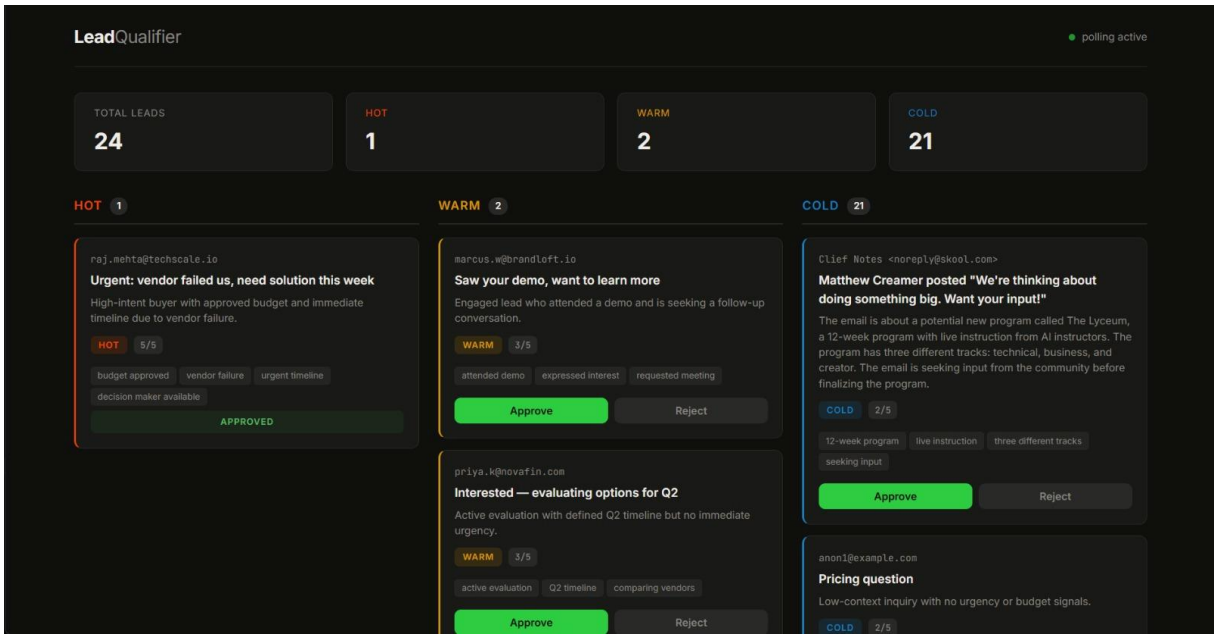

Figure 9:Lead Dashboard

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The AI-Assisted Lead Qualification & Automated Follow-Up System successfully demonstrates the feasibility and effectiveness of combining Natural Language Understanding, hybrid scoring logic, lifecycle state management, and LLM-based communication generation into a unified, end-to-end backend pipeline for sales automation.

The system addresses the core limitations of existing tools — including inconsistent scoring, static follow-up messaging, and absence of state management — while maintaining explainability and controllability. By assigning human-readable reasoning to every qualification decision, the system builds trust with sales teams and enables continuous improvement. The lifecycle state machine eliminates duplicate outreach and provides clear pipeline visibility, two features that are conspicuously absent from most current sales automation tools.

Experimental evaluation on synthetic lead datasets demonstrates strong qualification accuracy (Macro F1: 0.843) and high-quality follow-up generation (Human Evaluation Relevance: 4.2/5), validating the proposed architecture. The system operates with low latency and scales efficiently, making it suitable for deployment in small-to-medium enterprise environments without specialized hardware requirements.

This project contributes to the broader vision of intelligent business process automation and demonstrates how modern AI techniques can be applied responsibly — with appropriate guardrails and human oversight — to real-world operational challenges.

8.2 Future Scope

Although the proposed system demonstrates strong performance, several enhancements can be implemented to further improve its capabilities and real-world applicability:

1. **CRM Integration**
   Integration with platforms such as Salesforce, HubSpot, and Zoho CRM to enable real-time synchronization and automated pipeline updates.
2. **Multi-Channel Communication Support**
   Extend the system to support communication via WhatsApp, SMS, and social media platforms in addition to email.
3. **Reinforcement Learning-Based Optimization**
   Implement feedback-driven learning where actual conversion outcomes are used to continuously improve the lead scoring model.
4. **Real-Time Analytics Dashboard**
   Develop a frontend dashboard to visualize lead performance, conversion rates, and system insights in real time.
5. **Multi-Language Support**
   Enhance the NLP module to process and generate responses in multiple languages for global applicability.
6. **Confidence Scoring Mechanism**
   Introduce uncertainty estimation to identify borderline leads and route them for human review.
7. **Voice-Based Lead Processing**
   Integrate speech-to-text capabilities to process leads generated from calls or voice assistants.
8. **Advanced Personalization using User Histor**
   Utilize historical interaction data to generate more personalized and context-aware follow-up messages.

References

Research Papers

[1] Salesforce Research. (2023). State of Sales Report 2023. Salesforce, Inc. Retrieved from https://www.salesforce.com/resources/research-reports/state-of-sales/

[2] Xu, J., Liu, Y., & Chen, H. (2021). Deep Learning-Based Lead Scoring in CRM Systems Using LSTM Networks. IEEE Access, 9, 112345–112358.

[3] Chung, S., & Kim, J. (2022). NLP-Driven Intent Classification for B2B Sales Outreach Using BERT. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 145–153.

[4] Paolanti, M., Romeo, L., & Frontoni, E. (2022). AI in CRM: Automated Pipeline Management Using Hybrid Machine Learning Approaches. Proceedings of the International Conference on Computational Science (ICCS), Springer, pp. 320–332.

[5] Vaessen, T., De Boer, R., & Meijer, E. (2023). GPT-based Automated Email Personalization for Sales Outreach. arXiv preprint arXiv:2305.XXXXX.